

Relative path: scoring/risk_engine.py
Filename: risk_engine.py Extension: .py

```
"""
OASIS v2 – Layer 3: Impact & Risk Scoring
=====

Deliberately NOT a model (ARCHITECTURE.md Section 2 / design doc Section 5).
impact_score is transparent, whiteboard-able arithmetic:

    impact_score = w_anomaly * anomaly_score
                  + w_asset  * (asset_criticality / 5)
                  + w_mitre  * mitre_severity_weight

with w_anomaly + w_asset + w_mitre = 1. Default weights and the
recommended_action thresholds below are PLACEHOLDERS – reasonable starting
points, not tuned against any real incident data. Flagged as such here and
in the report's "known limitations" section, per the project's honesty norm.
"""
from __future__ import annotations

import json
from pathlib import Path
from typing import Optional

SCORING_DIR = Path(__file__).resolve().parent

# --- PLACEHOLDER: not empirically tuned, flagged per report Section 10 ---
DEFAULT_WEIGHTS = {"anomaly": 0.45, "asset": 0.30, "mitre": 0.25}
ACTION_THRESHOLDS = {"escalate": 0.70, "isolate": 0.40} # else -> acknowledge
# -----

def _load_json(name: str) -> dict:
    with open(SCORING_DIR / name) as f:
        return json.load(f)

class RiskEngine:
    def __init__(self, weights: Optional[dict] = None):
        self.weights = weights or DEFAULT_WEIGHTS
        self.mitre_map = _load_json("mitre_map.json")
        self.asset_criticality = _load_json("asset_criticality.json")
        self._default_criticality = self.asset_criticality.get("_default", 2)

    def _asset_criticality_for_session(self, resources_touched):
        if not resources_touched:
            return self._default_criticality
        return max(
            self.asset_criticality.get(r, self._default_criticality)
            for r in resources_touched
        )

    def _recommended_action(self, impact_score: float) -> str:
        if impact_score >= ACTION_THRESHOLDS["escalate"]:
            return "escalate"
        if impact_score >= ACTION_THRESHOLDS["isolate"]:
            return "isolate"
        return "acknowledge"

    def score(self, anomaly_event: dict, classified: dict, resources_touched: list) -> dict:
        """anomaly_event: dict with session_id/entity_id/anomaly_score/contributing_features
        classified: dict with anomaly_type/type_confidence (from classification/classifier.py)
        """
        anomaly_type = classified["anomaly_type"]
        if anomaly_type == "uncertain":
            mitre = {"technique_id": None, "technique_name": "Unclassified – below confidence threshold"}
            severity_weight = 0.35
        else:
            mitre = self.mitre_map.get(anomaly_type, {})
            severity_weight = mitre.get("severity_weight", 0.5)
        asset_crit = self._asset_criticality_for_session(resources_touched)

        w = self.weights
        impact_score = (
            w["anomaly"] * anomaly_event["anomaly_score"]

```

```

        + w["asset"] * (asset_crit / 5.0)
        + w["mitre"] * severity_weight
    )
    impact_score = round(min(1.0, max(0.0, impact_score)), 4)

    return {
        "session_id": anomaly_event["session_id"],
        "entity_id": anomaly_event["entity_id"],
        "anomaly_score": anomaly_event["anomaly_score"],
        "anomaly_type": anomaly_type,
        "type_confidence": classified["type_confidence"],
        "mitre_technique_id": mitre.get("technique_id"),
        "mitre_technique_name": mitre.get("technique_name"),
        "asset_criticality": asset_crit,
        "impact_score": impact_score,
        "contributing_features": anomaly_event["contributing_features"],
        "recommended_action": "acknowledge" if anomaly_type == "uncertain" else self._recommended_acti
on(impact_score),
    }

```